

Reviewer Pack — Minimal Order of Monomial Ideals and Communication Complexit...

Entry 91c080437dbf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 21:42:45 UTC

Minimal Order of Monomial Ideals and Communication Complexity Rank Correlation

Entry ID: 91c080437dbf **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 21:42:45 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Monomial Theory) **Field B** (complexity object): Communication Complexity

Statement:

For any Boolean function f , the communication complexity rank of f is linearly correlated with the minimal order of its associated monomial ideal I_f , such that $\text{comm_rank}(f) = \Theta(\min_order(I_f))$. Equivalently: for all instances φ of size n , if $\min_order(I_\varphi) \leq 2n/3$, then $\text{comm_rank}(\varphi) \leq 2$.

Rationale (proposer's reasoning):

Monomial ideals capture the essential algebraic structure of Boolean functions and have been used to study real polynomial problems. Their minimal order could reveal intricate relationships with communication complexity, potentially leading to new understanding of the computational power of communication protocols.

Taxonomy category: algebraic_geometry_monomial_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d51e7c3f71a46fcb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the randomly generated instances φ of size $n \leq 40$ satisfy $|\text{comm_rank}(\varphi) - \Theta(\min_order(I_\varphi))| \leq 3$, and at most one seed produces a value for $|\text{comm_rank}(\varphi) - \min_order(I_\varphi)| > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'algebraic geometry' AND 'monomial ideal' AND 'communication complexity' - 'minimal order monomial ideal' AND 'communication complexity rank' - 'algebraic geometry' AND 'minimal order of monomial ideal' AND 'rank correlation'

Top relevant hits considered: - [http://arxiv.org/abs/0810.2983v1] Polyhedral Methods in Numerical Algebraic Geometry - [http://arxiv.org/abs/1703.05377v4] Smith Ideals of Operadic Algebras in Monoidal Model Categories - [http://arxiv.org/abs/math/0504259v1] Multiplier ideal sheaves in complex and algebraic geometry - [http://arxiv.org/abs/1310.5598v2] The projective dimension of sequentially Cohen-Macaulay monomial ideals - [http://arxiv.org/abs/2009.08125v1] Support posets of some monomial ideals - [http://arxiv.org/abs/math/0305384v1] Orderings of Monomial Ideals - [http://arxiv.org/abs/2605.24303v1] Geometry of Neighborhoods of Minimal Rational Curves - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e} -linear maps

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_instance(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(instance):
        n = len(instance)
        # Simplified version of communication complexity rank calculation
        return n

    def min_order(monomial_ideal):
        # Simplified version of minimal order calculation
        return len(monomial_ideal)

    instances_tested = 0
    total_diff = 0
    max_n = 0

    for n in [5, 10, 15, 20, 30, 40]:
        if n > max_n:
```

```

max_n = n

for _ in range(5): # Test each size with 5 instances
    instance = generate_random_instance(n)
    comm_rank = communication_complexity_rank(instance)
    min_order_val = min_order(instance)

    diff = abs(comm_rank - (2 * min_order_val / 3))
    total_diff += diff

    instances_tested += 1

conjecture_holds = all(diff <= 3 for diff in [total_diff / instances_tested])

return {
    "metric_name": "communication_complexity_rank",
    "metric_value": total_diff / instances_tested,
    "instances_tested": instances_tested,
    "n_max": max_n,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_diff = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_diff} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_diff} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the expected time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13598
2	preregistration	ollama_remote	glm4:latest	0	0	9229
3	novelty	ollama_remote	glm4:latest	0	0	8467
4	novelty	ollama_remote	glm4:latest	0	0	9739
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14033
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29131
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13008
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7686
9	judge	ollama_remote	glm4:latest	0	0	44631

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 149522 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/91c080437dbf.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/91c080437dbf.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/91c080437dbf.tar.gz (if generated)